Lab 02 — Commented answers

Each answer follows the same pattern: the answer, the mechanism, the nuance or pitfall, an example you can check at the terminal.

Question 1 — Full names and short names

Answer.

Spelling	Full name	Podman
`nginx`	`docker.io/library/nginx:latest`	known alias → resolved without a question, message `Resolved "nginx" as an alias`
`bitnami/nginx`	`docker.io/bitnami/nginx:latest`	no alias → searches `unqualified-search-registries`; a single registry on Ubuntu (`docker.io`), so resolved; several on Fedora, so a **question**
`registry.mycompany.be:5000/base/nginx:1.25`	unchanged	full name: no resolution

The rule: if the first part (before the first `/`) contains a **dot** or a **colon** (port), or is `localhost`, it is a registry. Otherwise it is a namespace on the default registry.

Why. `mycompany.be` cannot be a Docker Hub username (dots are forbidden there), and a port only makes sense for a server. Docker applies that rule then completes silently; Podman applies the same rule but refuses to complete blindly, because `nginx` on `docker.io` and `nginx` on `registry.internal` may be two different images.

Nuance. `bitnami/nginx` is **not** an official image (namespace `bitnami`, not `library`), despite its name. And an image you build without a registry becomes `localhost/...`: a full name, with `localhost` as a fictitious "registry".

Example.

```
podman pull nginx 2>&1 | head -2          # Resolved "nginx" as an alias …
docker.io/library/nginx:latest
podman image inspect --format '{{.RepoTags}}' nginx
podman build -t api:1.0 . && podman images | grep api    # localhost/api  1.0
```

Question 2 — Same tag, different content

Answer. The `2.3` tag was **moved** in the meantime: someone rebuilt and re-pushed an image under the same name. A keeps the old image (no pull), B received the new one. You prove it by comparing digests; you avoid it by never reusing a published tag and by deploying by digest.

Why. A tag is a mutable pointer on the registry side. The `pull` compares the remote digest with the local one and only downloads if they differ. Nothing warns that a tag moved.

Nuance. The move may be unintentional: a pipeline that pushes `api:2.3` at every run on the release branch, or a `latest`. The base image may also have moved without your Dockerfile changing (`FROM eclipse-temurin:21-jre`): rebuilding "the same code" gives a different image.

Example.

```
# on A and on B:
podman image inspect --format '{{.Digest}}' myapp/api:2.3
# different digests -> the tag moved. Correct deployment:
podman pull registry.internal/myapp/api@sha256:9d0d1f1e…
```

Question 3 — 62 GB shown, disk intact

Answer. No. `SIZE` shows the **virtual** size of each image, shared layers included: common layers (JRE, Alpine, Debian) are counted in every image but stored once. `podman system df` gives the real usage. The files live in `~/.local/share/containers/storage` — inside the WSL distribution's virtual disk (`ext4.vhdx`), not in a Windows directory.

Why. The `overlay` driver stores each layer once, identified by content, and images are just lists of layers. Twelve Spring Boot images on the same JRE share its 180 MB.

Nuance. The WSL `vhdx` **grows** automatically but does not **shrink** by itself when you delete images: the space is freed on the Linux side, not on the Windows side, until you compact the disk (`wsl --shutdown` then `Optimize-VHD` or `diskpart`). A frequent surprise on a Windows workstation.

Example.

```
podman system df               # real SIZE and RECLAIMABLE
podman system df -v | head     # SHARED SIZE column per image
podman info --format '{{.Store.GraphRoot}}'   # /home/<you>/.local/share/containers/storage
```

Question 4 — The `rm` that removes nothing

Answer. They are wrong. `COPY` creates a layer that **contains** `credentials.json`. The `RUN … rm` creates a later layer containing a *whiteout* that hides the file. The final image contains both layers: the file is present, merely invisible from a container.

Why. Layers are immutable and additive. A layer cannot remove a file from a previous layer; it can only hide it. Anyone with the image can save it with `podman save`, extract the `COPY` layer and read the file.

Nuance. Even in a **single** `RUN` (`COPY` then `rm` in the same instruction), the `COPY` remains a separate instruction with its own layer. Only a *build secret* (`RUN --mount=type=secret`) or a multi-stage build (lab 05) avoids the presence of the file in the final image.

Example.

```
podman save --format oci-archive -o img.tar my-image:1.0
mkdir x && tar -xf img.tar -C x
for b in x/blobs/sha256/*; do tar -tf "$b" 2>/dev/null | grep -q credentials.json && echo
"present in $b"; done
```

Question 5 — 310 MB, 61 MB transferred

Answer. The registry already has the unchanged layers (JRE, system, dependencies); the `push` only transfers the new layers — the JAR and what follows — i.e. 61 MB. If everything were retransferred, the first layer of the image changes at every build: a `COPY . .` too early, or an instruction with variable content (date, version) before the heavy layers.

Why. Every layer has a digest. Before sending a blob, the client asks the registry whether it has it (`HEAD /v2/<repo>/blobs/<digest>`). Podman shows it less explicitly than Docker (no `Layer already exists`), but the transfer is instantaneous.

Nuance. Sharing between *repositories* of the same registry depends on its implementation (Harbor and Docker Registry do it via *cross-repository mount*). And an "unchanged" layer must be bit-for-bit identical: a `RUN apt-get update` without pinned versions produces a different layer at every build.

Example.

```
podman push --tls-verify=false localhost:5000/base/demo:1.1   # instantaneous: blobs already present
podman history my-api:2.0 --format 'table {{.Size}}\t{{.CreatedBy}}'  # spot the layer that changes
```

Question 6 — Two tags, one image, and a ghost

Answer. Two distinct images: `f3a1b9c02d11` (carrying the tags `2.0` and `1.9`) and `8b2c74e91a03` (unnamed). The `<none>` line is the *dangling* image: a tag (`2.0`, probably) was rebuilt and moved, the old image lost its name. `podman rmi api:1.9` removes **only the tag** (`Untagged: localhost/api:1.9`): the data remains, referenced by `2.0`. `localhost/` is the fictitious registry of any image built or tagged without a registry name.

Why. An `IMAGE ID` is the digest of the image configuration; two lines with the same ID are two names for one content. The data only goes with the last name.

Nuance. Do not confuse *dangling* (`<none>:<none>`, no tag at all) and *unused* (with a tag, but no container). A `podman image prune` without `-a` only removes the former.

Example.

```
podman rmi api:1.9                          # Untagged: localhost/api:1.9 (no Deleted)
podman images --filter dangling=true -q     # 8b2c74e91a03
podman rmi 8b2c74e91a03                     # Deleted: … this time the data goes
```

Question 7 — `exec format error`

Answer. The image was built for `linux/arm64` (Apple Silicon) and the server is `linux/amd64`: the kernel cannot execute the binary. Immediate fix: rebuild with `--platform linux/amd64` (QEMU emulation, slow but working). Durable fix: have the images built by **CI** on `amd64` agents, or produce multi-architecture images (`podman manifest`).

Why. A multi-arch tag is a manifest list; at `build`, the engine produces a manifest for the building machine's architecture. At `pull`, the server picks the `amd64` entry… which does not exist, so it receives `arm64`.

Nuance. *Official* images are almost all multi-arch, which hides the problem until the first home-made build. And the error can show up earlier: `podman run` on the Mac of an `amd64` image works (emulation), but 5 to 10 times slower.

Example.

```
podman image inspect --format '{{.Architecture}}' registry.internal/api:1.4   # arm64
podman build --platform linux/amd64 -t registry.internal/api:1.4 .
```

Question 8 — `save` versus `export`, and the OCI format

Answer. `save` exports an **image**: layers, manifest, configuration, tags. `export` exports the file system of a **container**, flattened into a single tree, without metadata. To carry a Spring Boot image to an isolated site, `save` is the right choice. With `export`, you lose `ENTRYPOINT`, `CMD`, `ENV`, `EXPOSE`, `USER`, `WORKDIR` — the imported image no longer knows how to start — as well as the layers (no more sharing or cache). `--format oci-archive` produces the same image in the standard OCI layout (`blobs/sha256/`, `index.json`) instead of Docker's historical format.

Why. `export` only sees the result of assembling the layers, like a `tar` taken from inside the container. The configuration lives in the image, not in the file system.

Nuance. `export` has a legitimate use: recovering a file system for analysis, or making a "flat" image from a hand-configured container (bad practice, but documented). The `docker-archive` format remains the most common; `oci-archive` is the one to use if the recipient is not Docker (Kubernetes via `ctr`, skopeo...).

Example.

```
podman save --format oci-archive -o api.tar my-api:1.0
podman load -i api.tar                          # Loaded image: localhost/my-api:1.0
podman export c1 | podman import - flat:1       # Config.Cmd = null
```

Question 9 — The second `pull`

Answer. The engine asked the registry for the tag's **manifest**, compared its digest with that of the local image, found them identical, and downloaded nothing else. The manifest is a few kilobytes: the network cost is one or two HTTP requests.

Why. Every layer and the manifest are content-addressed; the client knows exactly what it has. A pull is always differential, and in the limit, empty.

Nuance. Podman does not say "Image is up to date": it simply prints the image identifier. And "a few seconds" assumes a nearby registry; against Docker Hub, the request can take several seconds of latency, without downloading anything. Finally, that request counts towards Docker Hub's quota (question 10).

Example.

```
time podman pull alpine      # d529dd0c…  — a few seconds of network, zero layers transferred
```

Question 10 — `toomanyrequests`

Answer. Docker Hub caps anonymous `pull`s per IP address (and per account for authenticated users). CI agents all exit through the same public IP: the quota is shared by the whole company, hence "random" failures depending on the load of the moment. The two answers: a **pull-through cache** (internal registry that caches Docker Hub) and/or an **internal copy** of the base images into the company registry (`skopeo copy`), with Dockerfiles referencing that registry.

Why. Every `pull` queries at least the manifest, even when the image is already local. A CI that rebuilds a hundred times a day quickly exceeds the quota.

Nuance. Authenticating (`podman login docker.io`) raises the quota but does not cancel it, and puts a personal account in the CI. The internal copy has an extra advantage: you control *what comes in* (scan, validation), and you no longer depend on Docker Hub's availability.

Example.

```
skopeo copy docker://docker.io/library/node:22-alpine docker://registry.internal/base/node:22-alpine
# then in the Dockerfile: FROM registry.internal/base/node:22-alpine
```

Question 11 — HTTP versus HTTPS

Answer. Docker treats `localhost` (and `127.0.0.0/8`) as an *insecure* registry by default: it accepts HTTP without a word. Podman has no exception: every registry must present a valid TLS certificate. Two ways through: `--tls-verify=false` on the command, or a `[[registry]] location = "localhost:5000" insecure = true` entry in `registries.conf`. The second must never reach a versioned file or a server: it disables verification for **all** uses of that registry, silently.

Why. A registry without TLS can be impersonated by anyone on the network path (*man in the middle*) and return a booby-trapped image. On `localhost` the risk is low; Podman prefers you to say so rather than assume it.

Nuance. `--tls-verify=false` also disables **certificate** verification on a self-signed HTTPS registry — the good practice is rather to install the internal authority's certificate in `/etc/containers/certs.d/<registry>/ca.crt`.

Example.

```
podman push --tls-verify=false localhost:5000/base/demo:1.0      # explicit, visible in the history
# OR, for a development workstation only:
printf '[[registry]]\nlocation = "localhost:5000"\ninsecure = true\n' >>
~/.config/containers/registries.conf
```

Question 12 — `image is in use by a container`

Answer. A container (created from that image, even stopped) still exists; the image is its base layer. The engine refuses because removing the image would break that container. Clean way: list that container (`podman ps -a --filter ancestor=…`), remove it if its state is no longer needed, then `rmi`. `podman rmi -f` removes the tag and the image **and** the dependent container, without asking: you lose the logs, the writable layer and the ability to inspect the container — to save ten seconds.

Why. A container is image + writable layer. Without the image, the writable layer means nothing any more.

Nuance. Podman's message speaks of "external containers": those created by Buildah or another tool sharing the same storage, invisible in `podman ps`. `podman ps -a --external` shows them. Docker also refuses, but its message names the container identifier in the clear.

Example.

```
podman ps -a --filter ancestor=my-api:1.0 --format '{{.Names}} {{.Status}}'
podman logs <container> > incident.log     # we save what must be saved
podman rm <container> && podman rmi my-api:1.0
```

Question 13 — The `0B` layers

Answer. They are **metadata** instructions — `ENV`, `CMD`, `ENTRYPOINT`, `EXPOSE`, `LABEL`, `USER`, `WORKDIR` — which modify the image configuration without touching the file system. They appear in the history because every instruction leaves a trace, even an empty one. The 180 MB layer (a `RUN apt-get install`, a `COPY` of a JRE) is the only one that counts: `podman history` points at the line to optimise.

Why. An image is an array of instructions with, for some of them, an associated blob of files. The weight comes only from the blobs.

Nuance. A non-empty layer can hide a deletion: `RUN rm -rf /var/lib/apt/lists/*` in a separate `RUN` weighs almost `0B` but reclaims nothing. `history` shows the cost of each step; `podman image tree` additionally shows which layers come from the base image.

Example.

```
podman history nginx:alpine --format 'table {{.Size}}\t{{.CreatedBy}}'   # 12 lines at 0B, one at 50.7MB
podman image tree nginx:alpine
```

Question 14 — Three tagging strategies

Answer. (a) overwritten `latest`: **rollback impossible** (the old image has no name any more) and diagnosis impossible (no way to know which version was running). (b) `1.4.2`: immediate rollback to `1.4.1`; correct diagnosis if the tag is immutable, but two builds of `1.4.2` (a quick fix) can coexist without being distinguishable. (c) `1.4.2-b318-a9f3c21`: every image is unique, you get back to the exact commit and build; rollback to any earlier build. The cost is an unreadable name and retention to manage.

Why. Deployment and diagnosis need a **one-to-one** correspondence between a name and a content. Only (c) guarantees it by construction; (b) guarantees it by discipline; (a) forbids it.

Nuance. The three often coexist: CI pushes (c); a (b) tag is *added* to the same image once validated; `latest` only exists for developer convenience, never in a deployment manifest. And the deployment itself pins the digest.

Example.

```
podman tag registry.internal/api:1.4.2-b318-a9f3c21 registry.internal/api:1.4.2   # same image, second name
podman image inspect --format '{{.Digest}}' registry.internal/api:1.4.2           # what is actually deployed
```